ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

(ವಿ.ಟಿ.ಯು. ಅಡಿಯಲ್ಲಿನ ಸ್ವಾಯತ್ತ ಸಂಸ್ಥೆ)

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(Autonomous Under VTU)

Machine Learning – BAI503

Prof. Sanjay M Belgaonkar

Dept. of AI & ML

sanjaymb@bmsit.in

Logistic Regression

Logistic Regression

- Logistic regression is a statistical method used to model a binary dependent variable, where the outcome can take only two possible values, such as yes/no, success/failure, or 1/0.
- Unlike linear regression, which predicts a continuous output, logistic regression is used for classification problems.
- It predicts the probability that a given input belongs to a certain class by modeling a binary outcome using a logistic function (also called the sigmoid function).

How Logistic Regression Works

- Logistic regression calculates the probability of an outcome by using a linear combination of the input features and then applying a logistic function to squash this result between 0 and 1.
- This allows logistic regression to provide probabilities, which are then used to classify the data points into two classes.

Logistic (Sigmoid) Function

The logistic function used in logistic regression is:

$$f(x) = \frac{1}{1 + e^{-z}}$$

where $z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n$.

In this formula:

- z is the linear combination of input features x_i and their respective weights β_i.
- The function squashes z into a value between 0 and 1, which can be interpreted as the probability of the positive class.

Example

- Suppose you want to predict whether a student will pass (1) or fail (0) a class based on their hours of study. The dataset contains two columns: "**Hours of Study**" and "**Pass/Fail**" (1 if the student passed, 0 if not).

1. **Define the Model**: Logistic regression creates a model to estimate the probability that a student will pass based on the hours studied.

$$\text{Probability of Pass} = \frac{1}{1 + e^{-(\beta_0 + \beta_1 \cdot \text{Hours of Study})}}$$

2. **Fitting the Model**: Using training data, logistic regression finds the values of β_0 and β_1 that best separate the two classes.

3. **Make Predictions**: If the calculated probability is greater than 0.5, the student is classified as "Pass." If it's below 0.5, the student is classified as "Fail."

For example:

- A student who studied 5 hours might yield a probability of 0.8 (or 80%), so they are classified as "Pass."
- A student who studied 1 hour might yield a probability of 0.3 (or 30%), so they are classified as "Fail."

Interpretation

- Logistic regression doesn't directly predict the outcome but instead predicts the probability of a particular outcome.
- It’s widely used because it’s simple, interpretable, and efficient for binary classification tasks.

Exercise Questions

Ex. 1: Hours of Study and Pass/Fail Prediction

- Using the following dataset build a logistic regression model to predict whether a student will pass or fail based on hours studied.

Hours Studied	1	2	3	4	5	6	7	8	9	10
Pass (1) / Fail (0)	0	0	0	0	1	1	1	1	1	1

Ex. 2: Predicting Customer Churn

- Given a dataset with customer information such as "**Account Age**," "**Monthly Spend**," and "**Customer Complaints**," build a logistic regression model to predict whether a customer will churn (leave the service) or not.

Account Age	5	10	3	8	15	2	20	25	30	35
Monthly Spend	300	400	200	500	700	100	800	900	950	1000
Customer Complaints	0	1	0	2	3	1	0	4	1	3
Churn	0	0	0	0	1	1	1	1	0	1

Ex 3: Employee Attrition Prediction

- Use a dataset that includes features like "**Years at Company**," "**Satisfaction Score**," "**Salary Level**," and "**Promotion in Last 5 Years**" to predict if an employee is likely to leave the company.
- *Tasks*:
  - Fit a logistic regression model to classify employees as "Stay" or "Leave."
  - Interpret the coefficients to understand the relationship between features and the likelihood of attrition.
  - Use the model to identify employees who are likely to leave and suggest measures to reduce attrition based on model insights.

Years at Company	1	3	10	2	8	5	15	6	1	4
Satisfaction Score	0.8	0.4	0.6	0.9	0.3	0.5	0.2	0.7	0.8	0.6
Salary Level	Low	Medium	High	Low	High	Medium	High	Low	Medium	High
Promotion in last 5 Yrs	0	1	0	0	0	1	1	0	1	0
Attrition	0	1	0	0	1	0	1	0	1	0

- **Years at Company**: Number of years the employee has been with the company.
- **Satisfaction Score**: A score (e.g., between 0 and 1) indicating the employee's job satisfaction.
- **Salary Level**: Salary of the employee (can be treated as categorical, e.g., Low, Medium, High).
- **Promotion in Last 5 Years**: Binary indicator (1 if the employee was promoted in the last 5 years, 0 otherwise).
- **Attrition**: Target variable (1 if the employee left, 0 if they stayed).

Performance Metrics

1. **Accuracy**

- **Definition**: Accuracy is the ratio of correctly predicted instances (both passes and fails) to the total instances in the test data.
- **Formula**:

$$\text{Accuracy} = \frac{\text{True Positives} + \text{True Negatives}}{\text{Total Predictions}}$$

- **Interpretation**: It represents how often the model makes correct predictions. A high accuracy indicates that the model is generally correct, but it can be misleading if the data is imbalanced.

2. **Confusion Matrix**

- **Definition**: The confusion matrix is a table that shows the counts of true positives, true negatives, false positives, and false negatives.
- **Structure**:

	Predicted Pass	Predicted Fail
Actual Pass	True Positive (TP)	False Negative (FN)
Actual Fail	False Positive (FP)	True Negative (TN)

- **Interpretation**: It allows us to see the counts for each type of outcome and helps understand where the model might be making mistakes, especially in cases where some errors might have more significance than others.

3. **Classification Report (Precision, Recall, F1 Score)**

- A classification report provides several metrics for each class (e.g., pass and fail) in the form of precision, recall, and F1-score.
  - **Precision**: The proportion of positive predictions that are correct.

    $$\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$$

    Interpretation: High precision means that when the model predicts a student will pass, it is often correct. It's especially useful when false positives are costly.

  - **Recall (Sensitivity)**: The proportion of actual positives that are correctly identified.

    $$\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$$

    Interpretation: High recall indicates that the model correctly identifies most students who will pass. It's useful when missing a positive instance is costly.

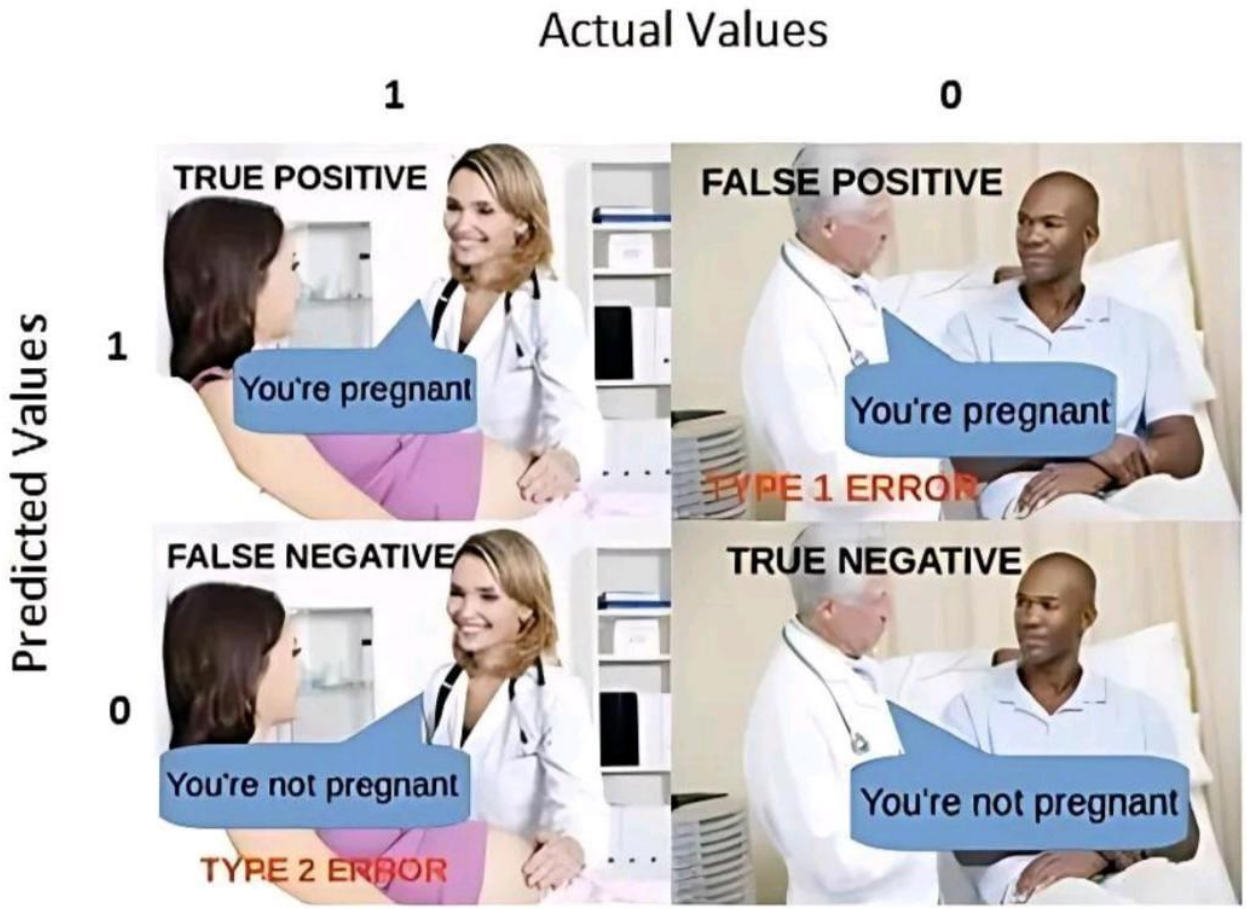
- **F1 Score**: The harmonic mean of precision and recall, balancing both metrics.

$$\text{F1 Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Interpretation: F1 Score is especially useful when you want a balance between precision and recall, particularly in imbalanced datasets.

4. **ROC-AUC (Receiver Operating Characteristic - Area Under the Curve)**

- **ROC Curve**: The ROC curve plots the true positive rate (recall) against the false positive rate at different probability thresholds.
- **AUC (Area Under the Curve)**: This is a single value summarizing the ROC curve. The AUC ranges from 0 to 1, with 1 indicating a perfect model and 0.5 representing a random guess.
- **Interpretation**: A higher AUC score indicates a better model. It's especially helpful when you need to compare models and their ability to distinguish between classes.



Predicted Values \ Actual Values	1	0
1	TP	FP
0	FN	TN

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \times precision \times recall}{precision + recall}$$

$$accuracy = \frac{TP + TN}{TP + FN + TN + FP}$$

Problem for Practice

Assuming we have the following confusion matrix from our model predictions:

	Predicted Pass	Predicted Fail
Actual Pass	15 (True Positive, TP)	3 (False Negative, FN)
Actual Fail	2 (False Positive, FP)	10 (True Negative, TN)

ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

(ವಿ.ಟಿ.ಯು. ಅಡಿಯಲ್ಲಿನ ಸ್ವಾಯತ್ತ ಸಂಸ್ಥೆ)

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(Autonomous Under VTU)

Machine Learning – BAI503

Prof. Sanjay M Belgaonkar

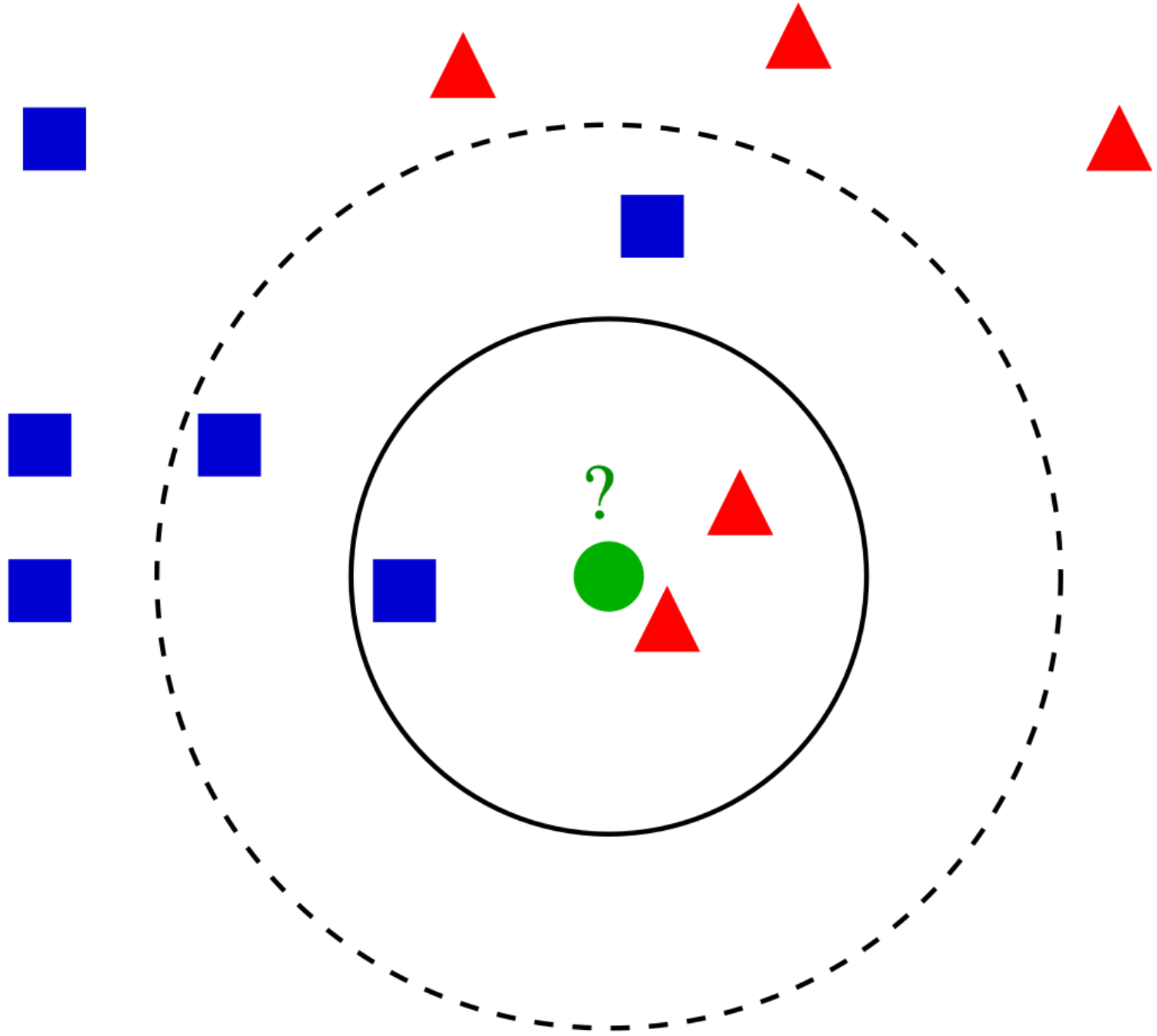
Dept. of AI & ML

sanjaymb@bmsit.in

k-Nearest Neighbours (k-NN) Algorithm

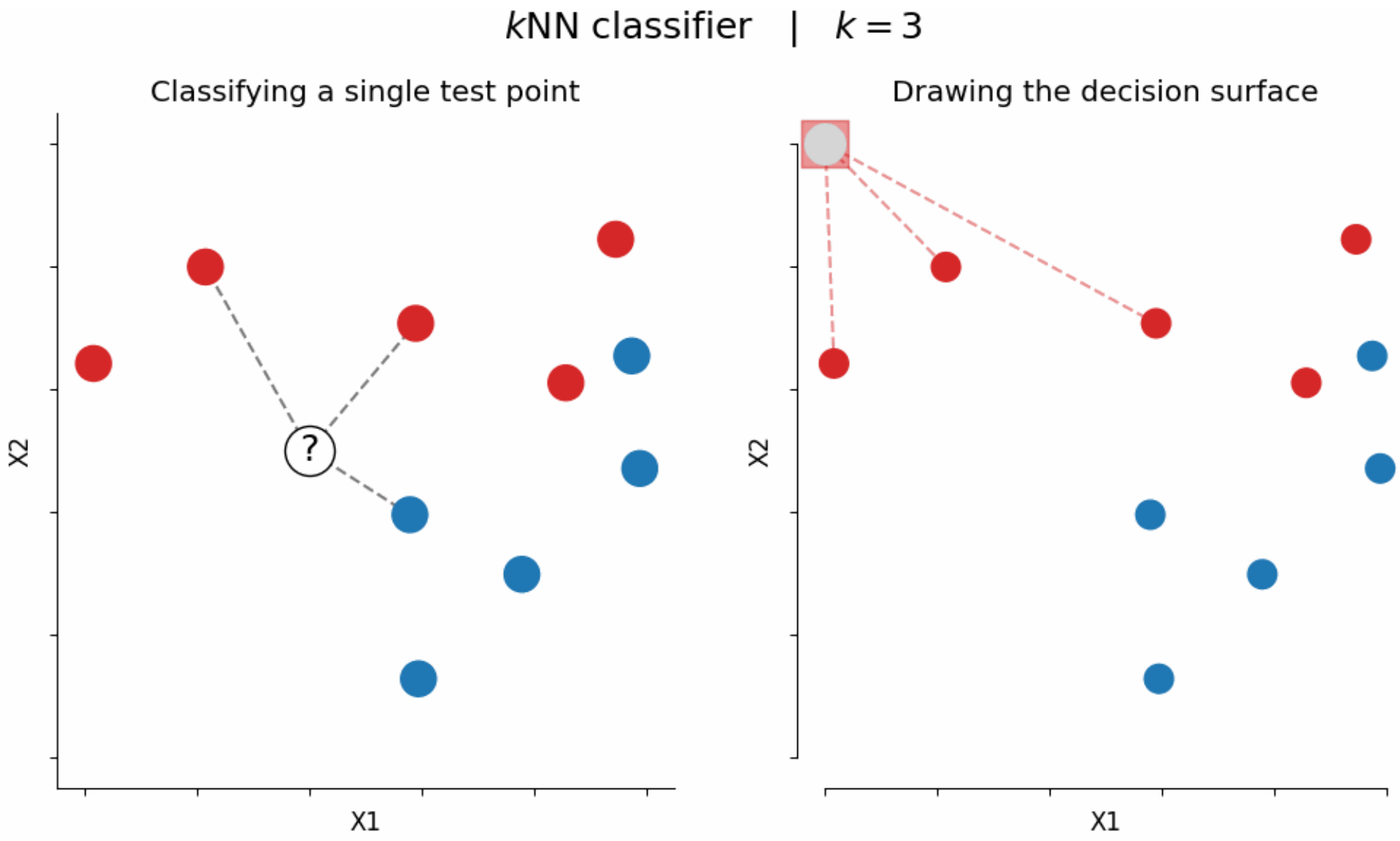
What is KNN (K-Nearest Neighbor) Algorithm in Machine Learning?

- The k-nearest neighbors (KNN) algorithm is a simple, supervised machine learning method that makes predictions based on how close a data point is to others.
- It's widely used for both classification and regression tasks because of its simplicity and popularity.
- Evelyn Fix and Joseph Hodges developed this algorithm in 1951, which was subsequently expanded by Thomas Cover.
- **Applications**: Commonly used in recommender systems, image and pattern recognition, text categorization, and more.



Working Principle of k-NN

- **Instance-Based Learning**: k-NN does not explicitly learn a model. Instead, it memorizes the training dataset, making predictions by evaluating the distance between new data points and points in the training set.
- **Classification Process**:
  - For a given data point (test sample), find the k nearest points (neighbors) in the training data.
  - Count the classes of these neighbors.
  - Assign the test sample to the class that is most common among the neighbors.
- **Regression with k-NN**: For regression tasks, k-NN predicts a value by averaging the values of the **k** nearest neighbors.


kNN classifier | k = 3
Classifying a single test point
?
X2
X1
Drawing the decision surface
X2
X1

Choosing the Value of "k" and Distance Metrics

Choosing the Value of `k`:

- If `k` is too small, the model becomes sensitive to noise (overfitting).
- If `k` is too large, the model may oversimplify and ignore smaller structures in the data (underfitting).
- Generally, an odd value of `k` is chosen to avoid ties, and values are often selected through cross-validation.

Distance Metrics

Distance Metrics:

- **Euclidean Distance**: Most commonly used for continuous data.

$$d(x, y) = \sqrt{\sum_{i=1}^{n} (x_i - y_i)^2}$$

- **Manhattan Distance**: Useful when we want to emphasize linear movement.

$$d(x, y) = \sum_{i=1}^{n} |x_i - y_i|$$

- **Minkowski Distance**: Generalization of Euclidean and Manhattan distances, with a parameter `p`.

$$d(x, y) = \left(\sum_{i=1}^{n} |x_i - y_i|^p \right)^{\frac{1}{p}}$$

- **Cosine Similarity**: Often used in text classification.

Strengths and Weaknesses of k-NN

- **Strengths**:
  - Simple and easy to understand.
  - No need for training, making it a “lazy learner.”
  - Effective with a clear, local structure in data, suitable for small datasets.
- **Weaknesses**:
  - **Computational Cost**: Requires storing the entire training dataset and can be slow during prediction, especially with large datasets.
  - **Memory Intensive**: Needs the whole dataset in memory.
  - **Sensitive to Outliers**: Outliers can disproportionately influence the classification of nearby points.
  - **Curse of Dimensionality**: High-dimensional spaces make it hard to determine “nearby” points as distances become less meaningful.

Example Problem

Consider a dataset of four data points, where each point belongs to one of two classes, "A" or "B". We want to classify a new point $(3, 3)$ based on the classes of its nearest neighbors.

Dataset

Point	X	Y	Class
1	1	2	A
2	2	3	A
3	3	1	B
4	6	5	B

New Point to Classify

We need to classify the point $(3, 3)$ using k-NN with different values of k.

Step-by-Step Solution

Step 1: Calculate Distances from the New Point to Each Point in the Dataset

Using **Euclidean Distance**:

$$\text{Distance} = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

1. **Distance to Point 1** $(1, 2)$:

$$\sqrt{(3-1)^2 + (3-2)^2} = \sqrt{4+1} = \sqrt{5} \approx 2.24$$

2. **Distance to Point 2** $(2, 3)$:

$$\sqrt{(3-2)^2 + (3-3)^2} = \sqrt{1+0} = \sqrt{1} = 1.0$$

3. **Distance to Point 3** $(3, 1)$:

$$\sqrt{(3-3)^2 + (3-1)^2} = \sqrt{0+4} = \sqrt{4} = 2.0$$

4. **Distance to Point 4** $(6, 5)$:

$$\sqrt{(3-6)^2 + (3-5)^2} = \sqrt{9+4} = \sqrt{13} \approx 3.61$$

Step 2: Sort Points by Distance

Now, we sort the points in ascending order of their distances to $(3, 3)$:

Point	X	Y	Class	Distance
2	2	3	A	1.0
3	3	1	B	2.0
1	1	2	A	2.24
4	6	5	B	3.61

Step 3: Choose Different Values of k and Classify the New Point

Case 1: $k = 1$

- The nearest neighbor is **Point 2**, which belongs to **Class A**.
- **Prediction**: Classify $(3, 3)$ as **Class A**.

Case 2: $k = 3$

- The 3 nearest neighbors are:
  - **Point 2 (Class A)**, **Point 3 (Class B)**, and **Point 1 (Class A)**
- Majority class among the nearest neighbors: **Class A** (2 votes for A, 1 vote for B).
- **Prediction**: Classify $(3, 3)$ as **Class A**.

Case 3: $k = 4$

- All points are considered as neighbors:
  - **Point 2 (Class A)**, **Point 3 (Class B)**, **Point 1 (Class A)**, and **Point 4 (Class B)**
- Class counts: 2 for A, 2 for B, resulting in a tie.

Handling a tie:

- In practice, the tie can be broken by selecting the class of the closest neighbor or choosing a smaller k.
- Here, if we select based on the closest point, $(3, 3)$ would be classified as **Class A**.

Summary

- With $k = 1$, $(3, 3)$ is classified as **Class A**.
- With $k = 3$, $(3, 3)$ is classified as **Class A**.
- With $k = 4$, there is a tie, but it can be resolved in favor of **Class A** if we look at the closest neighbor.

Ex. 1: Simple 2D Classification Problem

Given the dataset below, classify the point $(5, 4)$ using k-NN with $k = 1$, $k = 3$, and $k = 5$. Try using both **Euclidean** and **Manhattan distances** to observe differences in classification results.

Point	X	Y	Class
1	2	3	A
2	5	2	B
3	4	5	A
4	6	6	B
5	7	3	B
6	8	8	B
7	1	1	A

Exercise 2: Varying k with Different Distributions

Given the following dataset, classify the point $(4, 2)$ with k-NN using $k = 1$, $k = 3$, $k = 5$, and $k = 7$.

Point	X	Y	Class
1	1	3	A
2	3	4	A
3	6	1	B
4	7	3	B
5	5	6	A
6	2	2	A
7	8	4	B
8	6	5	B

Ex. 3: k-NN for Regression

- Given the following data points, predict the value for (4,5) using k-NN regression with k=3.

Point	X	Y	Value
1	2	3	50
2	3	4	60
3	5	6	80
4	7	3	70
5	4	4	65
6	6	5	75

ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

(ವಿ.ಟಿ.ಯು. ಅಡಿಯಲ್ಲಿನ ಸ್ವಾಯತ್ತ ಸಂಸ್ಥೆ)

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(Autonomous Under VTU)

Machine Learning – BAI503

Prof. Sanjay M Belgaonkar

Dept. of AI & ML

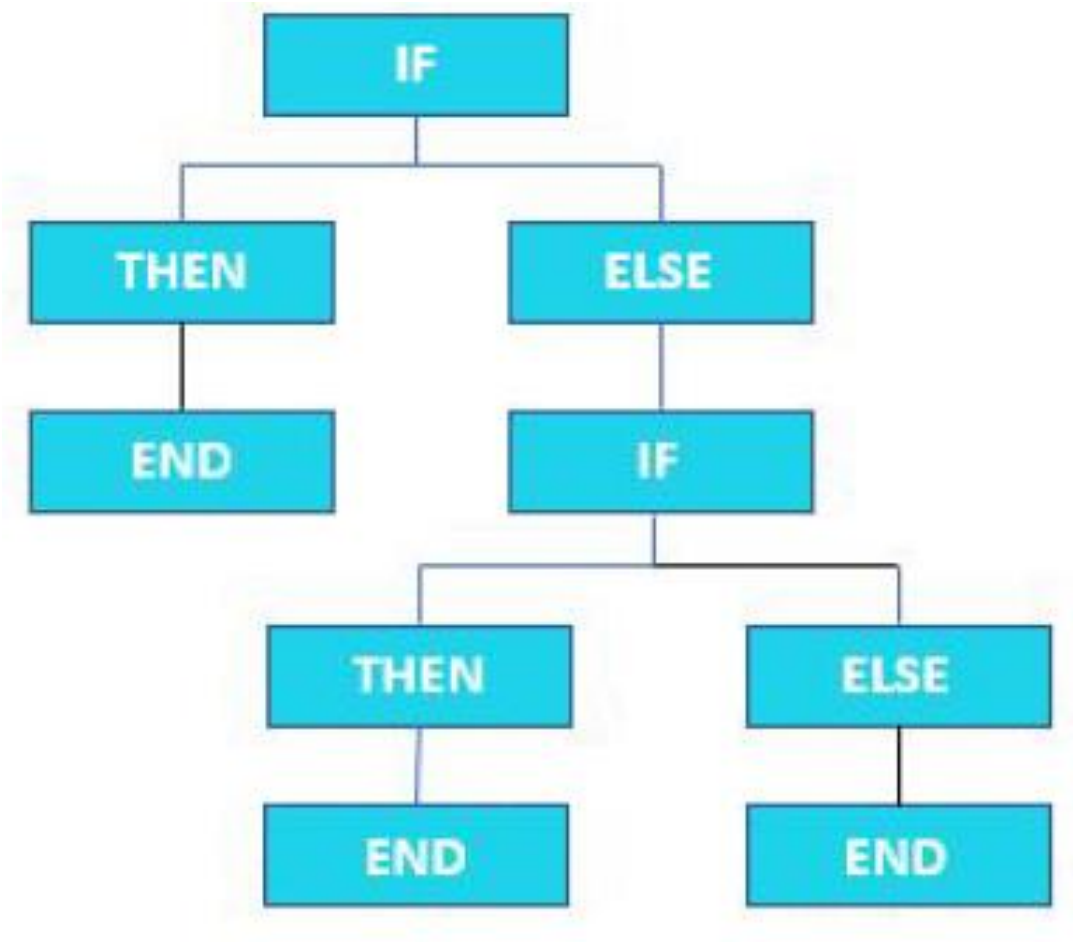
sanjaymb@bmsit.in

Introduction to Decision Trees

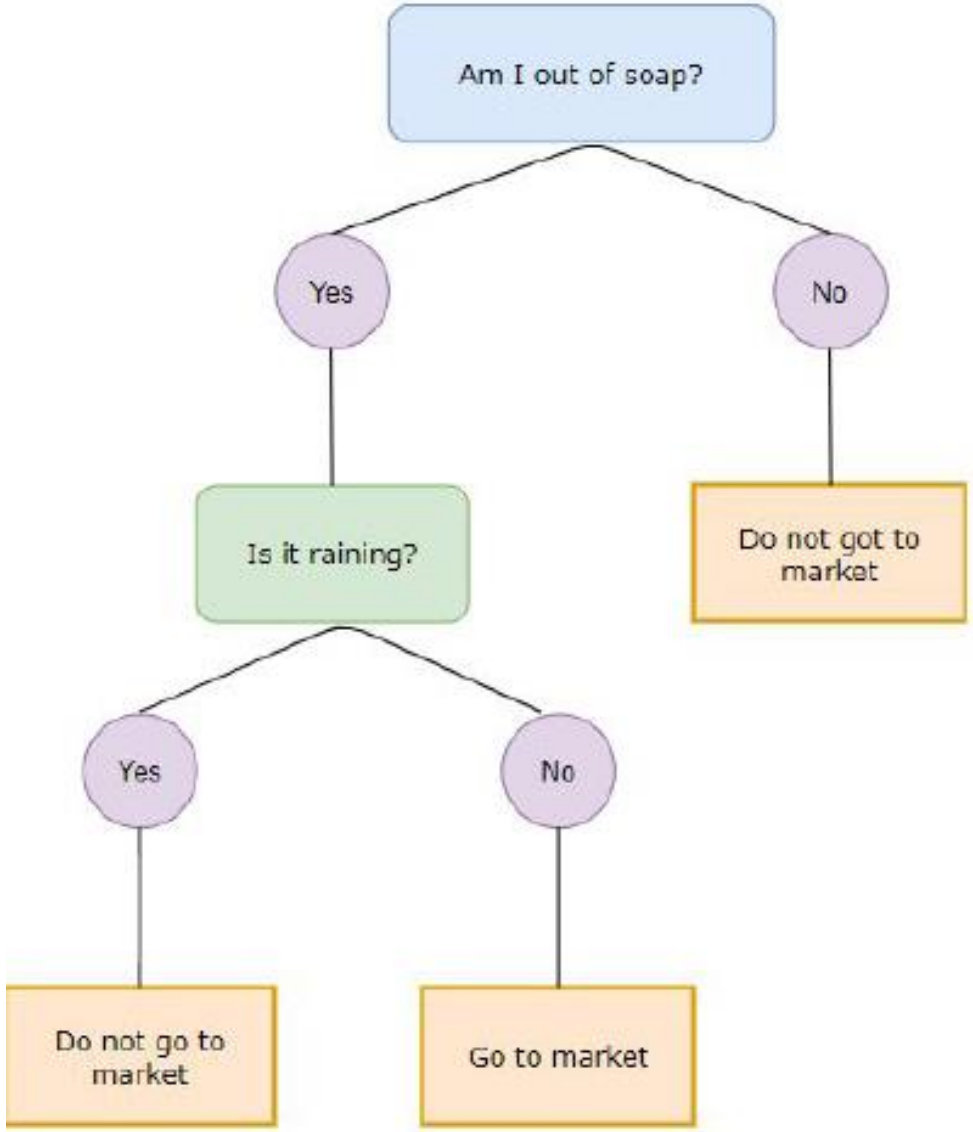
Introduction

- A decision tree is a machine learning algorithm used for classification and regression tasks.
- Decision Trees naturally represent the way we make decisions.
- It breaks down data by making a series of decisions based on feature values, much like a flowchart, to arrive at an outcome.
- It is a predictive model that resembles an upside-down tree.
- It is a supervised learning method, i.e., it has a fixed target variable, but unlike logistic Regression, it is not parametric.
- They are free to learn any functional form from the training data and do not have a fixed set of parameters to define the model.

- Decision trees can be considered a set of **if-then-else** statements.
- In other words, the process of making decisions involves asking questions with two or more possible outcomes.



- Let’s understand this decision-making process with the help of a simple decision tree example shown in the image given below.



Advantages of Decision Trees

- As you already know, variables are of two types: **categorical** and **continuous**. One of the major advantages of using a decision tree is that it can handle both categorical and continuous variables.
- Another advantage of decision trees is that they can be used for both classification and regression tasks.

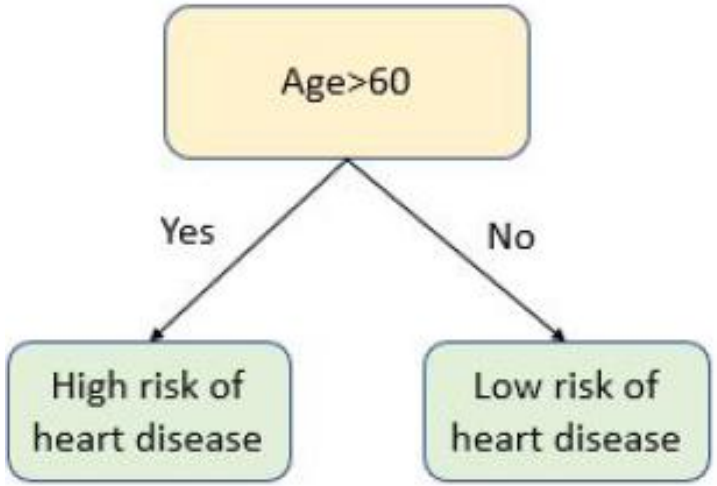
There are two types of decision trees:

1. Classification decision trees
2. Regression decision trees

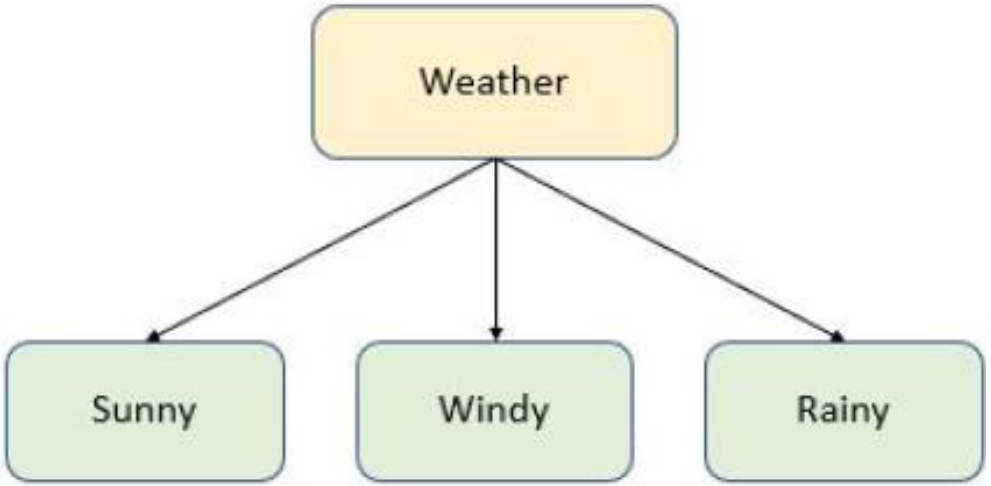
The other two types of decision trees based on the split-criteria are as follows:

1. Binary decision tree: The test splits variables into exactly two partitions.

2. Multiway decision tree: The test splits variables into more than two partitions.



Binary Decision Tree



Multi-way Decision Tree

The various components of a decision tree are as follows:

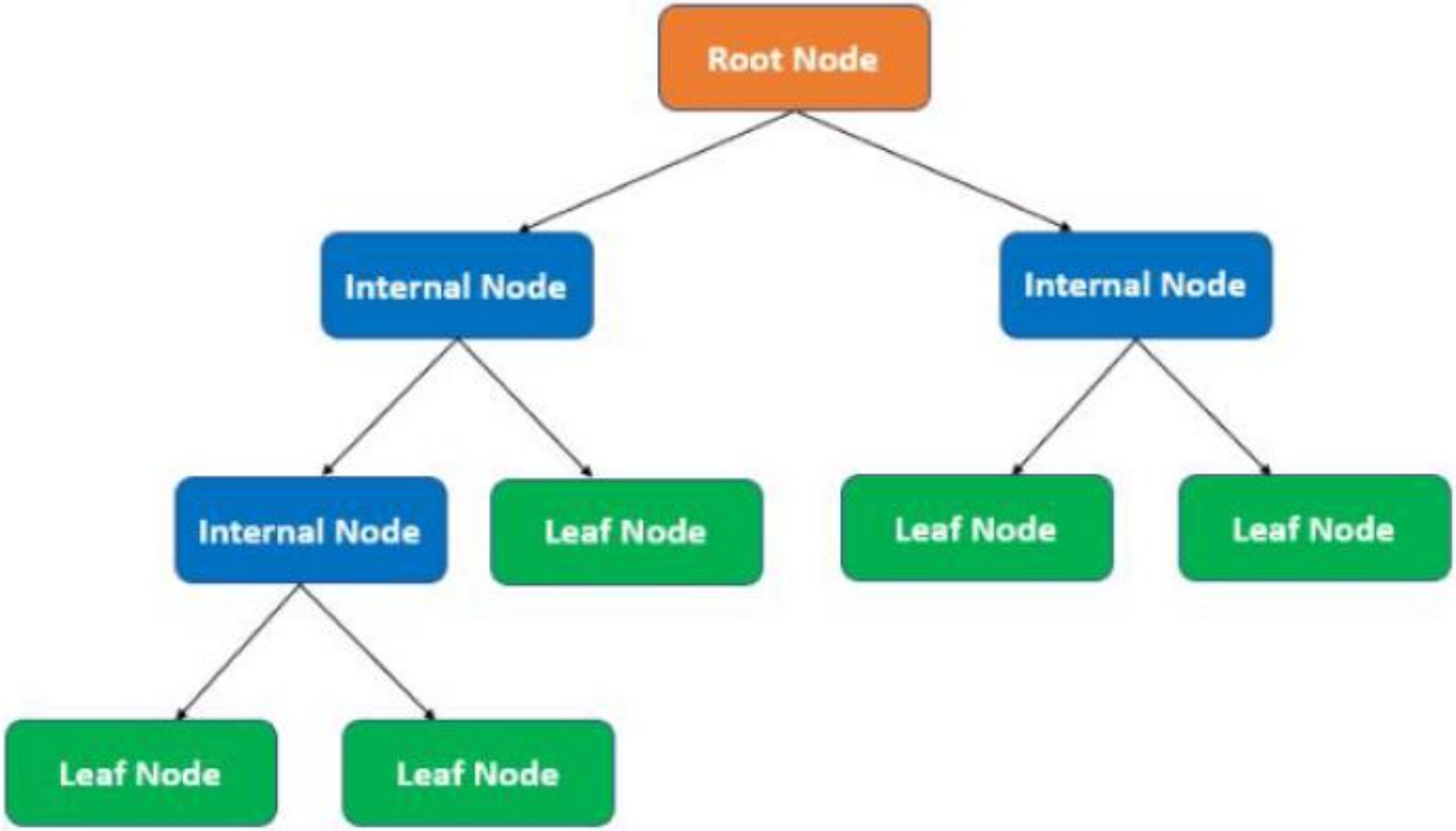
1. **Root node:** This is the starting/first node of a decision tree. It is further split into different nodes to form a tree. The arrows in a decision tree always point away from this node.

2. **Leaf/Terminal node:** The node that does not split into more nodes is known as a leaf node or a terminal node. The arrows in a decision tree always point towards this node.

3. **Internal node** : The intermediate nodes between the root and the leaf nodes are called the internal nodes.

4. **Branch/Subtree:** A subsection of the tree is called a branch or a subtree.

5. **Parent and child nodes:** A node that is divided into sub-nodes is called a parent node, and these sub-nodes can be referred to as the child nodes as they are children of the parent node.


Root Node
Internal Node
Internal Node
Internal Node
Leaf Node
Leaf Node
Leaf Node
Leaf Node
Leaf Node

Building Decision Trees

Constructing a decision tree involves the following steps:

- Recursive binary splitting/partitioning the data into smaller subsets.
- Selecting the best rule from a variable/ attribute for the split.
- Applying the split based on the rules obtained from the attributes.
- Repeating the process for the subsets obtained.
- Continuing the process until the stopping criterion is reached.
- Assigning the majority class/average value as the prediction.

Theoretical Implementation

- The theoretical implementation of a decision tree involves calculating specific metrics at each node to determine the best feature for splitting the data.
- The two common metrics are **Gini Impurity** and **Entropy** (used to calculate **Information Gain**).

Example

Imagine a scenario where we want to classify whether a person will play tennis based on weather conditions. Our dataset has three features:

- **Outlook** (Sunny, Overcast, Rainy)
- **Temperature** (Hot, Mild, Cool)
- **Wind** (Strong, Weak)

Our target variable is **Play Tennis** (Yes or No).

Outlook	Temperature	Wind	Play Tennis
Sunny	Hot	Weak	No
Sunny	Hot	Strong	No
Overcast	Hot	Weak	Yes
Rainy	Mild	Weak	Yes
Rainy	Cool	Weak	Yes
Rainy	Cool	Strong	No
Overcast	Cool	Strong	Yes
Sunny	Mild	Weak	No
Sunny	Cool	Weak	Yes
Rainy	Mild	Strong	No
Sunny	Mild	Strong	Yes
Overcast	Mild	Weak	Yes
Overcast	Hot	Strong	Yes
Rainy	Mild	Weak	Yes

Ex. 2: Buying a laptop

Price	Brand	RAM (GB)	Buy
High	Popular	16	Yes
High	Unpopular	8	No
Medium	Popular	8	Yes
Medium	Popular	4	Yes
Low	Unpopular	4	No
Low	Popular	8	Yes

Goal: Predict if a customer will buy a laptop based on **Price**, **Brand**, and **RAM**.

Steps:

- Calculate initial entropy for the **Buy** column.
- Calculate information gain for **Price**, **Brand**, and **RAM**.
- Select the best feature to split and continue until each path leads to a clear classification.

1. Calculate Entropy

Entropy quantifies the impurity or disorder of the dataset. For binary classification (Yes/No), the entropy E is given by:

$$E(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$

where:

- p_+ is the proportion of positive samples (Play Tennis = Yes),
- p_- is the proportion of negative samples (Play Tennis = No).

Initial Entropy Calculation

In our example dataset, there are **9 'Yes'** and **5 'No'** for the **Play Tennis** variable. Therefore,

$$p_+ = \frac{9}{14}, \quad p_- = \frac{5}{14}$$

Substituting these values, we get:

$$E(S) = -\left(\frac{9}{14}\right)\log_2\left(\frac{9}{14}\right) - \left(\frac{5}{14}\right)\log_2\left(\frac{5}{14}\right)$$

Calculating this gives an initial entropy $E(S) \approx 0.940$.

2. Calculate Information Gain for Each Feature

Information Gain (IG) helps us decide which feature to split on at each step. It is defined as:

$$IG(S, A) = E(S) - \sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} E(S_v)$$

where:

- S is the original dataset,
- A is the feature we are splitting on,
- S_v is the subset of S where feature A has value v,
- $E(S_v)$ is the entropy of subset S_v,
- $\frac{|S_v|}{|S|}$ is the proportion of samples in S_v.

We will calculate the Information Gain for each feature: **Outlook**, **Temperature**, and **Wind**.

Information Gain for "Outlook"

1. **Outlook = Sunny**: 5 samples → 2 'Yes', 3 'No'
   - $p_+ = \frac{2}{5}, \quad p_- = \frac{3}{5}$
   - $E(\text{Sunny}) = -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right) \approx 0.971$
2. **Outlook = Overcast**: 4 samples → 4 'Yes', 0 'No'
   - $p_+ = 1, \quad p_- = 0$
   - $E(\text{Overcast}) = 0$
3. **Outlook = Rainy**: 5 samples → 3 'Yes', 2 'No'
   - $p_+ = \frac{3}{5}, \quad p_- = \frac{2}{5}$
   - $E(\text{Rainy}) = -\frac{3}{5}\log_2\left(\frac{3}{5}\right) - \frac{2}{5}\log_2\left(\frac{2}{5}\right) \approx 0.971$

Using these values, we calculate the weighted entropy for **Outlook**:

$$E(S, \text{Outlook}) = \frac{5}{14} \cdot 0.971 + \frac{4}{14} \cdot 0 + \frac{5}{14} \cdot 0.971 \approx 0.693$$

The Information Gain for **Outlook** is:

$$IG(S, \text{Outlook}) = 0.940 - 0.693 = 0.247$$

Information Gain for "Temperature" and "Wind“

- Following the same process, calculate the **Information Gain** for **Temperature** and **Wind**.
- After performing these calculations, we determine which feature has the highest Information Gain.

ಬಿ.ಎಂ.ಎಸ್. ತಾಂತ್ರಿಕ ಮತ್ತು ವ್ಯವಸ್ಥಾಪನಾ ಮಹಾವಿದ್ಯಾಲಯ

(ವಿ.ಟಿ.ಯು. ಅಡಿಯಲ್ಲಿನ ಸ್ವಾಯತ್ತ ಸಂಸ್ಥೆ)

BMS INSTITUTE OF TECHNOLOGY & MANAGEMENT

(Autonomous Under VTU)

Machine Learning – BAI503

Prof. Sanjay M Belgaonkar

Dept. of AI & ML

sanjaymb@bmsit.in

Ensemble Learning & Random Forests

Ensemble Learning

- **Ensemble learning** is a powerful machine learning technique where multiple models, often referred to as "**weak learners**," are combined to solve a particular problem or improve model performance.
- The goal is to create a single, robust model that performs better than any individual model alone by leveraging the diversity and strengths of multiple models.

Ensemble Learning

- Ensemble learning is primarily used to improve the (classification, prediction, function approximation, etc.) performance of a model, or reduce the likelihood of an unfortunate selection of a poor one.
- Other applications of ensemble learning include assigning a confidence to the decision made by the model, selecting optimal (or near optimal) features, data fusion, incremental learning, non-stationary learning and error-correcting.

Types of Ensemble Methods

1. **Bagging (Bootstrap Aggregating)**
2. **Boosting**
3. **Stacking (Stacked Generalization)**

1. Bagging (Bootstrap Aggregating)

- In bagging, multiple models are trained on different subsets of the training data, each subset created by randomly sampling with replacement (bootstrapping).
- Each model independently makes predictions, and for classification, the final prediction is typically based on majority voting, while for regression, predictions are averaged.
- **Example**: **Random Forest**, which uses bagging with decision trees, is one of the most popular bagging methods.

2. Boosting

- Boosting involves training models sequentially, where each subsequent model tries to correct the errors of its predecessor.
- Models are added until no further improvement is observed or a predefined number of models is reached.
- Common boosting algorithms include **AdaBoost**, **Gradient Boosting**, and **XGBoost**.

3. Stacking (Stacked Generalization)

- Stacking is a technique that combines the predictions of multiple models by training a "**meta-model**" to make the final prediction.
- Each **base model** makes predictions, which are then used as inputs to train the **meta-model**.
- Stacking allows for greater flexibility since the base and meta-models can be of different types.

Advantages of Ensemble Learning

- **Improved Accuracy**: By combining models, ensemble learning can produce more accurate predictions.
- **Reduced Overfitting**: Certain ensemble methods, like bagging, help to reduce overfitting, especially with high-variance models like decision trees.
- **Robustness**: Ensembles are often more robust to noise and outliers since they capture a broader range of data patterns.

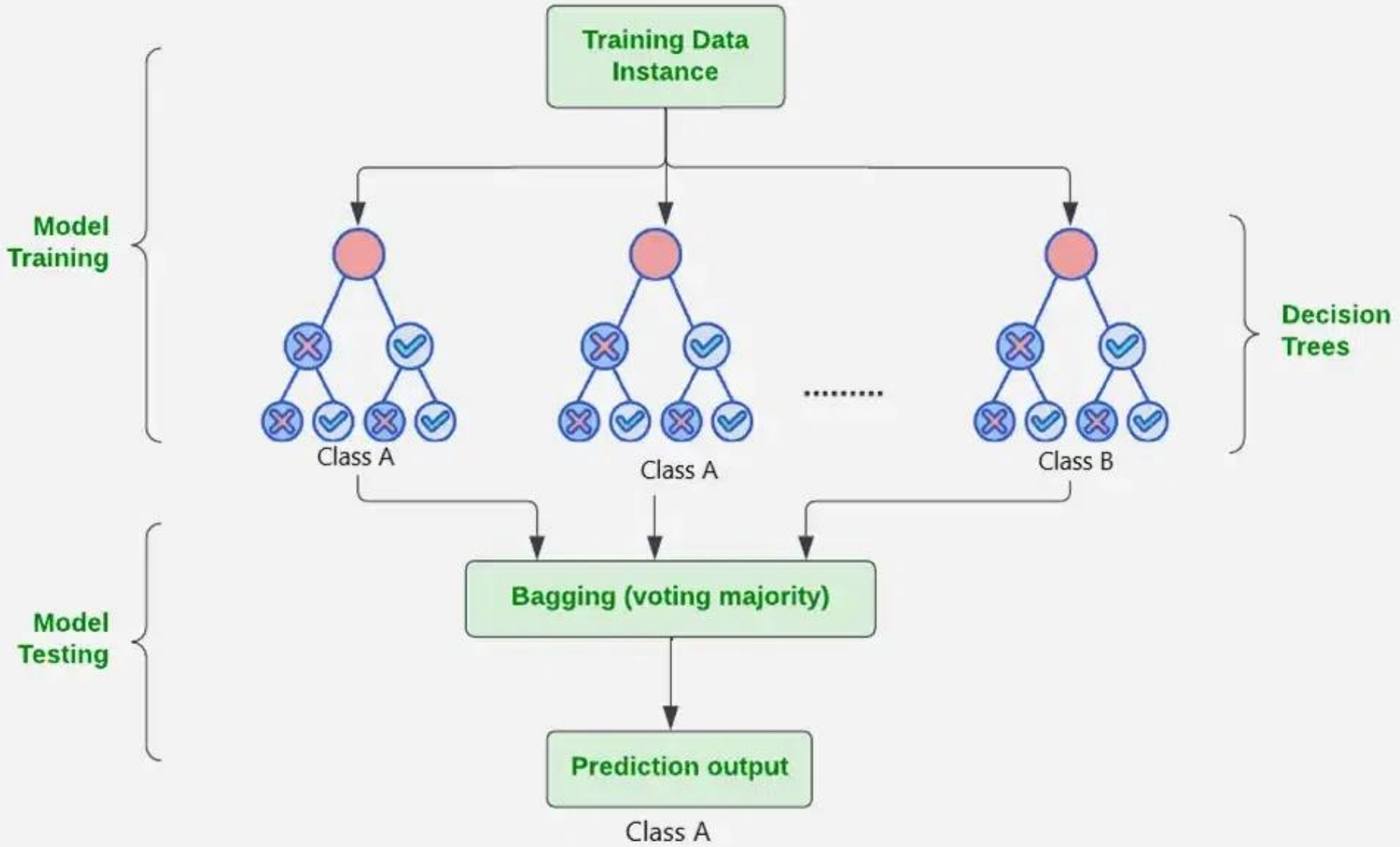
Disadvantages

- **Increased Complexity**: Ensemble models can be computationally expensive and harder to interpret due to the combination of multiple models.
- **Longer Training Times**: Because they involve multiple models, ensembles often require more training time and resources.

Ensemble of Decision Trees (Random Forest)

- Instead of relying on a single decision tree, **Random Forests** combine multiple decision trees to create a more robust model.
- So, It works by creating a number of Decision Trees during the training phase.
- Each tree is constructed using a random subset of the data set to measure a random subset of features in each partition.
- This randomness introduces variability among individual trees, reducing the risk of overfitting and improving overall prediction performance.

Random Forest Algorithm in Machine Learning



Let us see how does random forest work

1. **Random Sampling:** The Random Forest algorithm creates multiple decision trees, each trained on a different subset of the training data. This technique, known as **bootstrap sampling**, involves randomly sampling with replacement from the original dataset to create diverse subsets.
2. **Feature Randomness:** When each tree is trained, Random Forest only considers a random subset of features at each split. This randomness makes each tree unique and helps reduce correlation among trees.
3. **Combining Predictions:** For classification, each tree in the Random Forest votes for a class, and the majority vote is chosen as the final prediction. For regression, it takes the average of all tree outputs.

Example 1: Predicting Customer Purchase

Suppose we want to predict if a customer will buy a product. We have the following dataset:

Age	Income	Will Buy
22	High	Yes
45	Low	No
30	Medium	Yes
28	High	Yes
35	Low	No

Step 1: Create Multiple Trees

- Random Forest will create multiple decision trees, each trained on a different subset of this data.
- Some trees may focus more on "Age," while others may prioritize "Income."

Step 2: Individual Tree Predictions

- Tree 1 may predict **Yes** based on rules around **Income**.
- Tree 2 may predict **No** based-on **Age** thresholds.
- Tree 3 may predict **Yes** based on a combination of **Age** and **Income**.

Step 3: Aggregating Predictions

For a new customer with "Age = 29" and "Income = Medium," each tree in the forest gives its prediction:

- Tree 1 says: **Yes**
- Tree 2 says: **No**
- Tree 3 says: **Yes**

The Random Forest takes the majority vote (Yes) and predicts that the customer will buy the product.

Advantages of Random Forest

- **Reduces Overfitting:** Because each tree sees a different subset of data and features, the model is less likely to overfit.
- **Improves Accuracy:** By combining multiple trees, Random Forest is generally more accurate than a single decision tree.
- **Handles Missing Data Well:** The randomness and averaging effects make Random Forest robust to some missing data and noise.

Ex. 2. Predicting Admission into a University

GRE Score	GPA	Work Experience	Research Papers	Admission
320	3.5	2	Yes	Yes
310	3.2	1	No	No
325	3.8	3	Yes	Yes
300	2.9	0	No	No
315	3.4	2	Yes	Yes

Problem: Predict if a student will be admitted into a university program based on academic and extracurricular criteria.
Features: GRE Score, GPA, Work Experience (in years), Research Papers (Yes/No).
Target: Admission (Yes/No).